

Especificación API REST

Descripción

Colección de endpoints expuestos por el BFF (`bff-service`) y microservicios independientes. Los endpoints del BFF son puerta de entrada agregadora; los de microservicios se exponen directamente para comunicación inter-servicio.

Autenticación y seguridad

- **JWT:** Generado en `/auth/login`, enviado en header `Authorization: Bearer <token>`
 - **CORS:** Habilitado en BFF para `http://localhost:4173` (Frontend Vite)
 - **Filtro de seguridad:** En BFF se aplica a través de `SecurityConfig.java`
-

1. Endpoints del BFF-Service (Puerto 8084)

1.1 Autenticación (`/auth`)

POST /auth/register

Descripción: Registrar nuevo usuario

```
POST http://localhost:8084/auth/register
Content-Type: application/json

{
  "username": "nuevoUsuario",
  "password": "password123"
}
```

Response 200 OK:

```
"Usuario registrado exitosamente"
```

Response 400 BAD_REQUEST:

```
"Username y password son requeridos"
```

Response 409 CONFLICT:

```
"El usuario ya existe"
```

POST /auth/login

Descripción: Generar JWT token

```
POST http://localhost:8084/auth/login
Content-Type: application/json
```

```
{
  "username": "admin",
  "password": "admin123"
}
```

Response 200 OK:

```
{
  "username": "admin",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Response 401 UNAUTHORIZED:

```
"Credenciales inválidas"
```

Usuarios de prueba:

- Username: admin, Password: admin123
- Username: alumno, Password: password

1.2 Datos Académicos (/academico)

GET /academico/{estudianteId}

Descripción: Obtener datos consolidados de un estudiante (estudiante + asistencias + evaluaciones)

```
GET http://localhost:8084/academico/1
Authorization: Bearer <token>
```

Response 200 OK:

```
{
  "estudiante": {
    "id": 1,
    "run": "20.111.222-3",
    "nombre": "Juan Perez",
    "curso": "1-A"
  },
  "asistencias": [
    {
      "id": 101,
      "estudianteId": 1,
      "fecha": "2026-06-10",
      "presente": true
    }
  ],
  "evaluaciones": [
    {
      "id": 201,
      "estudianteId": 1,
      "materia": "Matemáticas",
      "nota": 7.5
    }
  ]
}
```

Response 404 NOT_FOUND:

```
"Estudiante no encontrado: {id}"
```

GET /academico/run/{run}

Descripción: Buscar estudiante por RUN (documento de identidad chileno)

```
GET http://localhost:8084/academico/run/20.111.222-3
Authorization: Bearer <token>
```

Response 200 OK: (idem a GET /academico/{estudianteId})

Response 404 NOT_FOUND:

```
"Estudiante con RUN 20.111.222-3 no encontrado en estudiante-service."
```

GET /academico/curso/{curso}

Descripción: Listar todos los estudiantes de un curso

```
GET http://localhost:8084/academico/curso/1-A
Authorization: Bearer <token>
```

Response 200 OK:

```
[
  {
    "id": 1,
    "run": "20.111.222-3",
    "nombre": "Juan Perez",
    "curso": "1-A"
  },
  {
    "id": 2,
    "run": "21.222.333-4",
    "nombre": "Ana Gomez",
    "curso": "1-A"
  }
]
```

POST /academico/estudiantes

Descripción: Crear nuevo estudiante

```
POST http://localhost:8084/academico/estudiantes
Content-Type: application/json
Authorization: Bearer <token>
```

```
{
  "run": "22.333.444-5",
  "nombre": "Carlos Rodríguez",
  "curso": "3-B"
}
```

Response 200 OK:

```
{
  "id": 999,
  "run": "22.333.444-5",
  "nombre": "Carlos Rodríguez",
  "curso": "3-B"
}
```

POST /academico/asistencias

Descripción: Registrar asistencia de un estudiante

```
POST http://localhost:8084/academico/asistencias
Content-Type: application/json
Authorization: Bearer <token>
```

```
{
  "estudianteId": 1,
  "presente": true,
  "fecha": "2026-06-12"
}
```

Response 200 OK:

```
{
  "status": "creado"
}
```

POST /academico/curso/{curso}/asistencia

Descripción: Registrar asistencia para todo un curso

```
POST http://localhost:8084/academico/curso/1-A/asistencia
Content-Type: application/json
Authorization: Bearer <token>
```

```
{
  "presente": true
}
```

Response 200 OK:

```
[
  {"id": 1, "status": "creado"},
  {"id": 2, "status": "creado"}
]
```

POST /academico/evaluaciones

Descripción: Crear evaluación para un estudiante

```
POST http://localhost:8084/academico/evaluaciones
Content-Type: application/json
Authorization: Bearer <token>
```

```
{
  "estudianteId": 1,
  "materia": "Historia",
  "nota": 6.0
}
```

Response 200 OK:

```
{
  "id": 100
}
```

PUT /academico/evaluaciones/{id}

Descripción: Actualizar nota de una evaluación

PUT http://localhost:8084/academico/evaluaciones/100
Content-Type: application/json
Authorization: Bearer <token>

```
{
  "nota": 7.5
}
```

Response 200 OK:

```
{
  "id": 100,
  "materia": "Historia",
  "nota": 7.5
}
```

DELETE /academico/estudiantes/{id}

Descripción: Eliminar estudiante

DELETE http://localhost:8084/academico/estudiantes/999
Authorization: Bearer <token>

Response 204 NO_CONTENT: (sin body)

DELETE /academico/evaluaciones/{id}

Descripción: Eliminar evaluación

DELETE http://localhost:8084/academico/evaluaciones/100
Authorization: Bearer <token>

Response 204 NO_CONTENT: (sin body)

2. Endpoints de Microservicios (comunicación inter-servicio)

2.1 Estudiante-Service (Puerto 8081)

GET /estudiantes

Response: List<Estudiante>

GET /estudiantes/{id}

Response: Estudiante

GET /estudiantes/run/{run}

Response: Estudiante

GET /estudiantes/curso/{curso}

Response: List<Estudiante>

POST /estudiantes

Body: {run, nombre, curso} Response: Estudiante (201 CREATED)

DELETE /estudiantes/{id}

Response: 204 NO_CONTENT

2.2 Asistencia-Service (Puerto 8082)

GET /asistencias/estudiante/{estudianteId}

Response: List<Asistencia>

POST /asistencias

Body: {estudianteId, fecha, presente} Response: Asistencia

PUT /asistencias/{id}

Body: {presente} Response: Asistencia

DELETE /asistencias/{id}

Response: 204 NO_CONTENT

2.3 Evaluacion-Service (Puerto 8083)

GET /evaluaciones/estudiante/{estudianteId}

Response: List<Evaluacion>

POST /evaluaciones

Body: {estudianteId, materia, nota} **Response:** Evaluacion

PUT /evaluaciones/{id}

Body: {nota} **Response:** Evaluacion

DELETE /evaluaciones/{id}

Response: 204 NO_CONTENT

3. Códigos HTTP comunes

Código	Significado
200	OK - Operación exitosa
201	CREATED - Recurso creado
204	NO_CONTENT - Operación exitosa sin respuesta
400	BAD_REQUEST - Parámetros inválidos
401	UNAUTHORIZED - Sin autenticación
404	NOT_FOUND - Recurso no encontrado
409	CONFLICT - Conflicto (ej: duplicado)
500	INTERNAL_SERVER_ERROR - Error del servidor

4. DTOs principales

EstudianteDTO

```
{
  "id": 1,
  "run": "20.111.222-3",
  "nombre": "Juan Perez",
  "curso": "1-A"
}
```

AcademicoDTO

```
{
  "estudiante": { /* EstudianteDTO */ },
  "asistencias": [ /* Array de Asistencia */ ],
  "evaluaciones": [ /* Array de Evaluacion */ ]
}
```

AuthResponse

```
{
  "username": "admin",
  "token": "eyJhbGciOiJIUzI1NiI..."
}
```

5. Ejemplos de curl

```
# Login
curl -X POST http://localhost:8084/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admin123"}'

# Obtener datos de estudiante (requiere token)
curl -X GET http://localhost:8084/academico/1 \
  -H "Authorization: Bearer <token>"

# Crear estudiante
curl -X POST http://localhost:8084/academico/estudiantes \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d '{"run":"22.333.444-5","nombre":"Carlos","curso":"3-B"}'

# Registrar asistencia
curl -X POST http://localhost:8084/academico/asistencias \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d '{"estudianteId":1,"presente":true}'
```